

Software Architecture Laboratory

# Laboratory No. 2

## Concurrent Programming

Greyhound Race Simulation with Java 21

**Students:**

Andersson David Sánchez Méndez

Cristian Santiago Pedraza Rodríguez

Elizabeth Correa Suárez

Juan Sebastian Ortega Muñoz

**Instructor:** Professor Javier Ivan Toquica Barrera

**Course:** Software Architecture (ARSW)

**Institution:** Escuela Colombiana de Ingeniería Julio Garavito

February 6, 2025

## Contents

|                                                          |          |
|----------------------------------------------------------|----------|
| <b>1 Objective</b>                                       | <b>3</b> |
| <b>2 Problem Description</b>                             | <b>3</b> |
| 2.1 Application Architecture . . . . .                   | 3        |
| <b>3 Theoretical Framework</b>                           | <b>3</b> |
| 3.1 Thread Synchronization with join() . . . . .         | 3        |
| 3.2 Critical Regions and Synchronized . . . . .          | 3        |
| 3.3 Wait/Notify Pattern . . . . .                        | 3        |
| <b>4 Implementation</b>                                  | <b>4</b> |
| 4.1 Solution 1: Thread Completion with join() . . . . .  | 4        |
| 4.2 Solution 2: Critical Region Protection . . . . .     | 4        |
| 4.3 Solution 3: Pause/Continue Synchronization . . . . . | 4        |
| <b>5 Experiments and Results</b>                         | <b>4</b> |
| 5.1 Race Execution Evidence . . . . .                    | 4        |
| 5.2 Final Results Display . . . . .                      | 5        |
| 5.3 Testing and Coverage . . . . .                       | 6        |
| <b>6 Synchronization Flow</b>                            | <b>6</b> |
| <b>7 Analysis</b>                                        | <b>6</b> |
| 7.1 Correctness Verification . . . . .                   | 6        |
| 7.2 Thread Safety Analysis . . . . .                     | 6        |
| 7.3 Performance Considerations . . . . .                 | 6        |
| <b>8 Methodology</b>                                     | <b>6</b> |
| 8.1 Test Environment . . . . .                           | 6        |
| 8.2 Testing Strategy . . . . .                           | 6        |
| <b>9 Conclusions</b>                                     | <b>7</b> |
| 9.1 Lessons Learned . . . . .                            | 7        |
| <b>10 References</b>                                     | <b>7</b> |

## 1 Objective

This laboratory focuses on implementing thread synchronization mechanisms in a Java Swing application that simulates a greyhound race. The simulation involves multiple concurrent threads representing racing dogs, requiring proper coordination to ensure correct behavior and data consistency.

The specific goals include:

- **Thread Completion:** Ensure the main thread waits for all racing dog threads to finish before displaying final results.
- **Critical Regions:** Protect shared resources (arrival registry) from race conditions using synchronization.
- **Pause/Continue Control:** Implement inter-thread communication for pausing and resuming the race simulation.

## 2 Problem Description

The greyhound race simulator presents three main concurrency challenges:

### Race Condition Warning

1. **Premature Results:** The UI displays "final results" before all dogs have finished racing, showing incomplete data.
2. **Race Condition:** Multiple dogs finishing simultaneously may corrupt the arrival registry due to unsynchronized access.
3. **Control Problem:** No mechanism exists to pause and resume the race in progress.

### 2.1 Application Architecture

The application follows a layered architecture:

| Package | Responsibility           |
|---------|--------------------------|
| app     | Application entry point  |
| ui      | Swing GUI components     |
| threads | Racing dog threads       |
| domain  | Arrival registry model   |
| control | Race synchronization     |
| util    | Random number generation |

Table 1: Application package structure

## 3 Theoretical Framework

### 3.1 Thread Synchronization with join()

The `join()` method blocks the calling thread until the target thread terminates. This provides a barrier synchronization mechanism:

Listing 1: Thread Barrier with `join()`

```
1 Thread[] workers = new Thread[N];
2 // Start all threads
3 for (Thread t : workers) {
```

```
4     t.start();
5 }
6 // Wait for all to complete
7 for (Thread t : workers) {
8     t.join();
9 }
10 // All threads have finished
```

### Important Note

The `join()` method throws `InterruptedException`, requiring proper exception handling to maintain thread interrupt status.

### 3.2 Critical Regions and Synchronized

A **critical region** is a code section that accesses shared resources and must be executed atomically. Java's `synchronized` keyword provides mutual exclusion:

Listing 2: Synchronized Method

```
1 public synchronized void
2     registerArrival(String name) {
3     // Only one thread can execute
4     // this method at a time
5     int position = arrivals.size() + 1;
6     arrivals.add(name + " : " + position);
7 }
```

The intrinsic lock (monitor) ensures:

- **Mutual Exclusion:** Only one thread holds the lock.
- **Visibility:** Changes are visible to other threads.
- **Atomicity:** The method executes as a single unit.

### 3.3 Wait/Notify Pattern

The `wait()`/`notifyAll()` mechanism enables inter-thread communication through a shared monitor:

Listing 3: Monitor Pattern

```
1 public class RaceControl {
2     private boolean paused = false;
3
4     public synchronized void pauseRace() {
5         paused = true;
6     }
7
8     public synchronized void resumeRace() {
9         paused = false;
10        notifyAll(); // Wake waiting threads
11    }
12
13    public synchronized void
14        checkPausePoint() throws
15        InterruptedException {
16        while (paused) {
17            wait(); // Release lock and wait
18        }
19    }
20 }
```

### Wait/Notify Semantics

- `wait()`: Releases lock, enters waiting state
- `notify()`: Wakes one waiting thread
- `notifyAll()`: Wakes all waiting threads
- Use `while` loop to guard against spurious wakeups

## 4 Implementation

### 4.1 Solution 1: Thread Completion with join()

The main thread must wait for all dog threads to complete before displaying final results.

**Listing 4:** MainCanodromo.java - join() Implementation

```
1 // Start all dog threads
2 carrilSeleccionado.iniciarCarrera();
3
4 // Wait for all to finish
5 for (Galgo galgo : galgos) {
6     try {
7         galgo.join();
8     } catch (InterruptedException e) {
9         Thread.currentThread()
10            .interrupt();
11     }
12 }
13
14 // Now safe to show results
15 printArrivalOrder();
```

**Why it works:** The join() call blocks the main thread until each individual Galgo thread terminates. Only after all threads have completed does execution continue to display the final results.

### 4.2 Solution 2: Critical Region Protection

The arrival registry is a shared resource accessed by multiple dog threads simultaneously.

**Listing 5:** ArrivalRegistry.java - Synchronized Method

```
1 public class ArrivalRegistry {
2     private List<String> arrivals =
3         new ArrayList<>();
4
5     public synchronized void
6         registerArrival(String name) {
7         int position = arrivals.size() + 1;
8         arrivals.add(
9             name + " finished in position " +
10            position);
11     }
12
13     public synchronized List<String>
14         getArrivals() {
15         return new ArrayList<>(arrivals);
16     }
17 }
```

**Why it works:** The synchronized keyword ensures that:

1. Only one thread can register an arrival at a time.
2. The position calculation and list addition are atomic.
3. No two dogs can get the same finishing position.

### 4.3 Solution 3: Pause/Continue Synchronization

The RaceControl class implements the monitor pattern for pausing and resuming the race.

**Listing 6:** RaceControl.java - Full Implementation

```
1 public class RaceControl {
2     private boolean paused = false;
3
4     public synchronized void pauseRace() {
5         this.paused = true;
6     }
7
8     public synchronized void resumeRace() {
9         this.paused = false;
10        notifyAll();
11    }
```

```
12
13    public synchronized void
14        checkPausePoint()
15        throws InterruptedException {
16        while (paused) {
17            wait();
18        }
19    }
20
21    public synchronized boolean
22        isPaused() {
23        return paused;
24    }
25 }
```

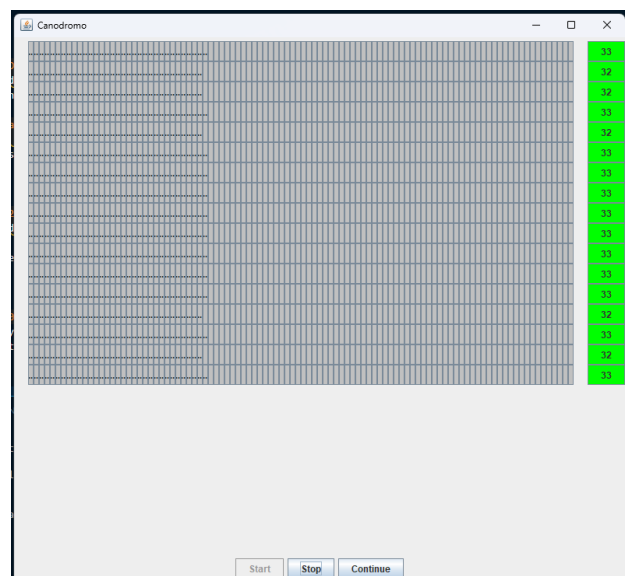
Each dog thread calls checkPausePoint() in its run loop:

**Listing 7:** Galgo.java - Pause Check Integration

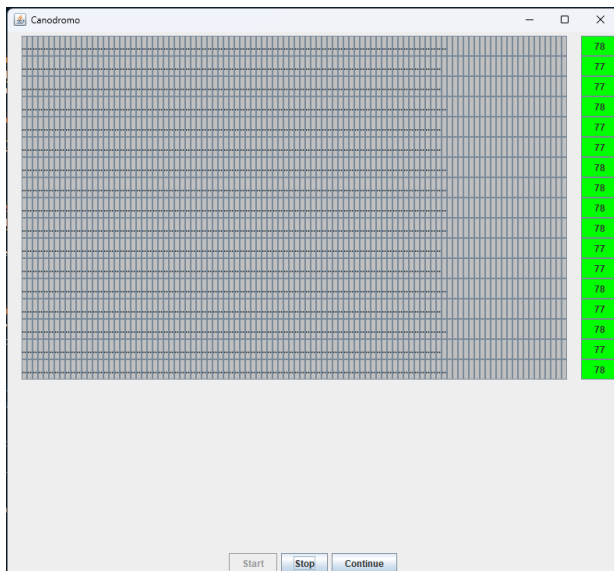
```
1 public void run() {
2     while (!finished) {
3         try {
4             // Check if race is paused
5             raceControl.checkPausePoint();
6
7             // Continue running
8             advance();
9             Thread.sleep(randomDelay());
10        } catch (InterruptedException e) {
11            Thread.currentThread().interrupt();
12            break;
13        }
14    }
15 }
```

## 5 Experiments and Results

### 5.1 Race Execution Evidence

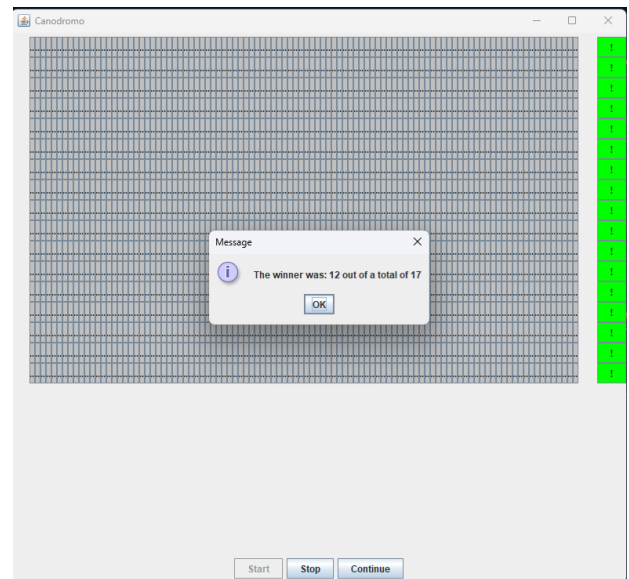


**Figure 1:** Race in progress: All greyhounds running concurrently in their lanes.

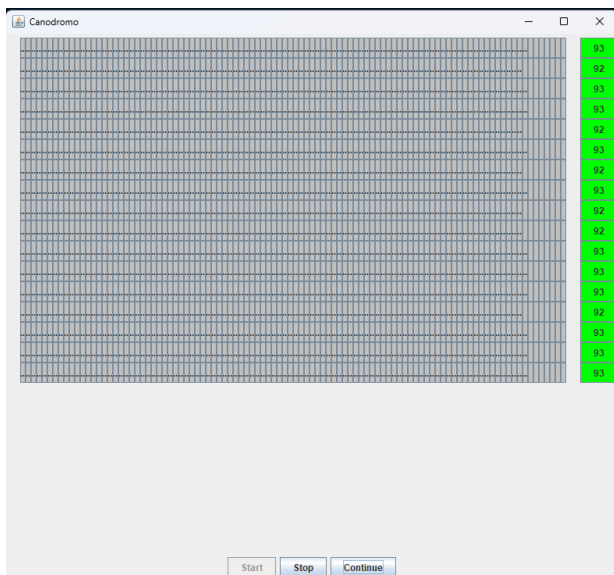


**Figure 2:** Race paused: All dogs frozen in place after pressing the Pause button.

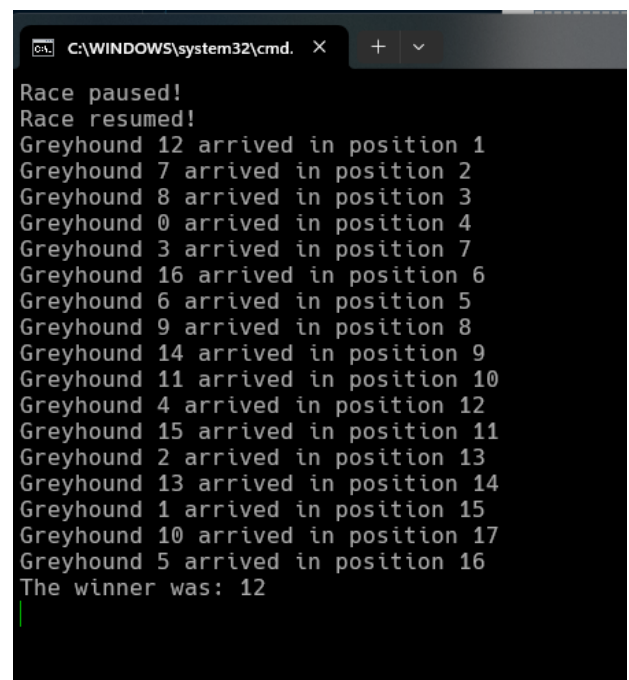
## 5.2 Final Results Display



**Figure 4:** Correct final results: All dogs shown with proper finishing positions (Part 1).



**Figure 3:** Race resumed: Dogs continue from their paused positions after pressing Continue.



**Figure 5:** Correct final results: Complete arrival order displayed (Part 2).

5.3 Testing and Coverage

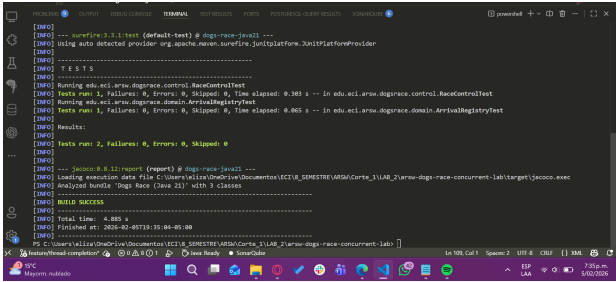


Figure 6: All unit tests passing: Testing thread-safe arrival registration and race control.

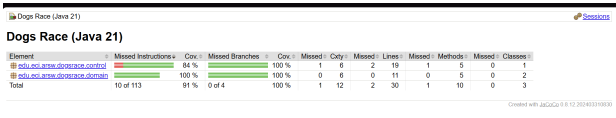


Figure 7: JaCoCo code coverage report: Coverage targets met for domain and control packages.

6 Synchronization Flow

The following sequence illustrates the interaction between the UI thread, race control, and dog threads during pause/resume operations:

- 1. **Normal Operation:** Dog threads run independently, advancing their position.
- 2. **Pause Request:** UI thread calls `pauseRace()`, setting the flag.
- 3. **Pause Point:** Each dog thread enters `wait()` at the next `checkPausePoint()`.
- 4. **Resume Request:** UI thread calls `resumeRace()`, clearing the flag.
- 5. **Wake Up:** `notifyAll()` wakes all waiting dog threads.
- 6. **Continue:** Dog threads exit the wait loop and resume racing.

Thread States During Synchronization

- **RUNNABLE:** Dog actively racing
- **WAITING:** Dog paused via `wait()`
- **BLOCKED:** Dog waiting to acquire monitor lock
- **TERMINATED:** Dog has finished the race

7 Analysis

7.1 Correctness Verification

The implementation guarantees correctness through:

- **Barrier Synchronization:** `join()` ensures all threads complete before results display.
- **Mutual Exclusion:** `synchronized` prevents concurrent registry modifications.

- **Proper Signaling:** `notifyAll()` ensures all waiting threads are awakened.
- **Spurious Wakeup Protection:** `while` loop guards the `wait()` call.

7.2 Thread Safety Analysis

| Class           | Issue            | Solution     |
|-----------------|------------------|--------------|
| ArrivalRegistry | Race condition   | synchronized |
| RaceControl     | Coordination     | wait/notify  |
| MainCanodromo   | Premature output | join()       |

Table 2: Thread safety solutions by component

7.3 Performance Considerations

- **Lock Contention:** The `synchronized` method in `ArrivalRegistry` creates a bottleneck only at race finish, which is acceptable for this use case.
- **Wait Efficiency:** Threads calling `wait()` release the CPU, avoiding busy-waiting.
- **Scalability:** The solution scales with additional dog threads without modification.

8 Methodology

8.1 Test Environment

Experiments were conducted using the following configuration:

| Component      | Specification  |
|----------------|----------------|
| Java Version   | OpenJDK 21     |
| Build Tool     | Maven 3.9+     |
| Test Framework | JUnit 5        |
| Coverage Tool  | JaCoCo 0.8.12  |
| Code Analysis  | SonarCloud     |
| CI/CD          | GitHub Actions |

Table 3: Development environment

8.2 Testing Strategy

Unit tests verify thread-safe behavior:

Listing 8: Concurrent Test Example

```
@Test
void testConcurrentArrivals()
    throws InterruptedException {
    ArrivalRegistry registry =
        new ArrivalRegistry();
    int numThreads = 100;
    Thread[] threads = new Thread[numThreads];

    for (int i = 0; i < numThreads; i++) {
        final String name = "Dog-" + i;
        threads[i] = new Thread(() ->
            registry.registerArrival(name));
        threads[i].start();
    }

    for (Thread t : threads) {
        t.join();
    }

    assertEquals(numThreads,
        registry.getArrivals().size());
}
```

## 9 Conclusions

### Key Findings

1. **join() for Completion:** The `join()` method provides a simple and effective barrier synchronization for waiting on thread completion.
2. **synchronized for Safety:** The `synchronized` keyword protects critical regions, ensuring atomic operations on shared data structures.
3. **wait/notify for Communication:** The monitor pattern enables efficient inter-thread communication without busy-waiting.
4. **Design Patterns:** Separating synchronization logic into `RaceControl` follows the Single Responsibility Principle and improves testability.
5. **Testing Importance:** Concurrent code requires specific testing strategies to verify thread safety under contention.

### 9.1 Lessons Learned

- Always use `while` loops with `wait()` to handle spurious wakeups.
- Prefer `notifyAll()` over `notify()` when multiple threads may be waiting.
- Handle `InterruptedException` properly to preserve interrupt status.
- Synchronization should be at the smallest scope necessary to minimize contention.

## 10 References

1. Goetz, B. et al. (2006). *Java Concurrency in Practice*. Addison-Wesley Professional.
2. Oracle. Java 21 Documentation - Concurrency. <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Thread.html>
3. Lea, D. (2000). *Concurrent Programming in Java: Design Principles and Patterns*. Addison-Wesley.
4. Herlihy, M., Shavit, N. (2012). *The Art of Multiprocessor Programming*. Morgan Kaufmann.